

System design as a conversation

Week 6, Lecture 1 · Landing the Offer: SWE Job Search for Senior CS Students

Autumn 2026

What we will cover

- The interview rubric: what a senior engineer is actually scoring
- Requirements clarification: functional vs. non-functional, why you ask first
- Capacity estimation: QPS, storage, and bandwidth math that earns credibility
- The order of operations: the sequence that separates good from great candidates
- Common failure modes: what candidates do that costs them the offer

Part 1: the rubric

What an interviewer is actually scoring

- System design is not a quiz. There is no single correct answer.
- The interviewer is scoring your process, not your diagram.
- ByteByteGo (2023): “The goal of the first step is not to solve the problem. It is to agree on what problem you are actually solving.”
- The rubric rewards: clarifying requirements, stating tradeoffs, adapting when pushed back on.

The four-step framework (ByteByteGo, 2023)

1. **Understand the problem and establish scope.** Ask before you draw.
2. **Propose a high-level design.** Get the interviewer to nod before you go deep.
3. **Design deep dive.** Pick one component and go one level deeper together.
4. **Wrap up.** Summarize tradeoffs, name what you would improve with more time.

Why candidates lose points before drawing a box

- Most candidates jump to a database and API server within 90 seconds.
- They are solving the wrong problem, or the right problem at the wrong scale.
- ByteByteGo (2023): requirements clarification is where most candidates lose points before writing a single component.
- A senior engineer treats the whiteboard like a conversation, not a presentation.

Part 2: requirements clarification

Functional vs. non-functional requirements

- **Functional:** what the system does. User-facing behaviors. Can be stated as API calls.
- **Non-functional:** how the system performs. Scale, latency, durability, consistency.
- Example for a URL shortener:
 - Functional: shorten a URL, redirect a short code, expire old links.
 - Non-functional: 100 million redirects per day, p99 latency under 100ms, 99.9% uptime.

Questions that change the design

- How many daily active users? (determines database tier and cache size)
- Read-heavy or write-heavy? (determines sharding strategy)
- Does the system need to be globally distributed? (introduces CDN and replication)
- What consistency guarantee does the user need? (strict vs. eventual)
- What is the retention policy? (determines storage and indexing decisions)

The scope boundary: what you are NOT building

- Explicitly state what is out of scope. This is not weakness. It is engineering judgment.
- System Design Primer (Martin, 2017): “Out of scope” saves you from designing a distributed operating system when the question is a URL shortener.
- Example: “I am not designing analytics on click counts for this session. We can add that in the deep-dive if time allows.”
- Naming scope boundaries shows the interviewer you understand tradeoffs.

Part 3: capacity estimation

The math that earns credibility

- Back-of-envelope estimation is a signal: you have built or thought hard about real systems.
- You do not need precision. You need an order-of-magnitude answer that informs a decision.
- A number that is wrong by 2x is fine. A design with no numbers is a red flag.

URL shortener: a worked estimate

Scale: 100M redirects/day (reads)

Read-to-write ratio: 100:1 $\Rightarrow$ 1M new short URLs/day (writes)

Reads per second: $100\text{M} / 86,400\text{s} \approx 1,160$ reads/sec

Writes per second: $1\text{M} / 86,400\text{s} \approx 12$ writes/sec

Storage (one year):

1M writes/day * 365 days = 365M URLs

Each row: 500 bytes (short code + long URL + metadata)

$365\text{M} * 500 \text{ bytes} \approx 182 \text{ GB/year}$

Martin (2017): "Estimates help clarify what kind of storage and servers you need."

How the estimate connects to the design

- 1,160 reads/sec with $p99 < 100\text{ms}$: a single database cannot handle this without a cache.
- 12 writes/sec: not the bottleneck. Write path can be simpler than read path.
- 182 GB/year: fits on a single modern server, so sharding is optional in year one.
- Each number pushes a design decision. That is the point of the estimate.

Part 4: order of operations

The sequence that separates good from great

1. Requirements (functional + non-functional, scope boundary)
2. Capacity estimation (QPS, storage, bandwidth)
3. API design (endpoint signatures, response types)
4. Data model (schema sketch, SQL vs. NoSQL decision, sharding key)
5. High-level diagram (all components named, data flow labeled)
6. Deep dive (one component in depth with the interviewer)

Why this order matters

- Requirements before design: you cannot choose a database until you know the access pattern.
- Capacity before components: you cannot size a cache until you know the read QPS.
- API before schema: the API shape reveals what queries the database must answer.
- High-level before deep-dive: the interviewer cannot guide a deep-dive on a diagram they haven't seen.
- Skipping a step is visible. Interviewers fill in the gaps with the worst-case assumption.

Part 5: common failure modes

The five most common mistakes

1. **Jumping to a solution.** Drawing a diagram before stating requirements. Costs: 10-15 points.
2. **Vague non-functionals.** "It should be fast and scalable" without numbers. Costs: credibility.
3. **Defending instead of adapting.** When the interviewer pushes back, restating the original design. Signals: low collaborative ceiling.
4. **No explicit tradeoffs.** Choosing Redis without saying why or what it costs. Signals: surface-level knowledge.
5. **Skipping the deep dive.** Running out of time before going one level deeper. Costs: the signal that separates new-grad from senior.



The goal of the first step is not to solve the problem. It is to agree on what problem you are actually solving. Requirements clarification is where most candidates lose points before writing a single box.

Take-aways

1. System design interviews score process, not correctness. A legible process with explicit tradeoffs beats a clever diagram with no reasoning.
2. Ask requirements questions before drawing. Functional requirements name what the system does. Non-functional requirements name numbers.
3. Back-of-envelope capacity estimates earn credibility and drive design decisions. Show the arithmetic.
4. Follow the order of operations: requirements, capacity, API, data model, diagram, deep dive.
5. State tradeoffs explicitly. Choosing Redis or PostgreSQL without a reason is not a design decision, it is a guess.

What's next

- **Reading:** Week 6 reading walks through a URL shortener end to end with worked capacity math, SQL vs. NoSQL analysis, and a component diagram
- **Lecture 2 (this week):** Storage, caching, and scale: SQL vs. NoSQL, three caching layers, sharding patterns, queues
- **Section (this week):** System design lab: sketch a URL shortener or rate limiter, then peer critique
- **HW 4 (out this week):** System design write-up for a well-known product, due week 8